ENGINEERING REVIEW

SEO Command Center Engineering Review

Strict architecture, code quality, security, performance, and product scalability assessment

Generated on
2026-04-09

Repository root
B:\AI SEO Agent\seo-command-center

1. Project Understanding

SEO Command Center is a multi-workspace SEO operations platform that combines AI-assisted SEO chat, audit storage, website strategy memory, backlink automation, recurring reporting, content publishing, and operational monitoring in one Next.js application.

Current System Flow

- Users authenticate with credential-based NextAuth and enter a protected dashboard.
- Project profiles store per-website strategy memory, Google/Search/CMS connection settings, and publishing preferences.
- Audits, reports, and backlink campaigns are created through App Router APIs backed by Prisma services.
- AI chat and compare flows resolve the most relevant project profile,

Tech Stack

- Next.js 16 App Router
- React 19 + TypeScript
- Prisma ORM
- SQLite for local development and Postgres-ready schema workflow for production
- NextAuth credentials authentication
- Tailwind CSS + Radix UI
- Zod validation
- Resend email delivery
- Playwright browser testing

route requests across multiple providers, and persist full session history.
- Background work such as recurring reports, audits, and autonomous backlink cycles runs through a durable queue and drain worker.
- Ops and analytics surfaces expose queue health, AI provider behavior, delivery status, and incidents in plain language.

- Multi-provider AI routing for Claude, ChatGPT, Gemini, Grok, and Groq

Main Modules

- Dashboard and Ops Center
- AI Chat and AI Analytics
- Projects / website memory
- Backlink Agent and outreach workflow
- Reports, delivery logs, and schedules
- Content operations and CMS publishing
- Queue processing and worker orchestration
- Shared service, validation, auth, response, and observability layers

2. Deep Code Review

1. Recurring report schedules could be duplicated after worker crashes

Severity: Critical

Problem: Claiming a due schedule previously placed a lease on the current run but did not advance nextRunAt immediately.

Why it matters: If a worker crashed after claiming but before marking the run complete, the same schedule could become due again and create duplicate client reports or owner digests.

Resolution: The schedule claim path now advances nextRunAt at claim time, stores processingRunAt separately, and clears or restores the correct run timestamp on completion/failure.

Impacted files: lib/services/report-automation-service.ts, lib/server/job-queue.ts

2. Queue events were not scoped tightly enough per user

Severity: High

Problem: Recent job events were effectively global, which allowed one operator's queue outcomes to appear in another operator's notifications or Ops view.

Why it matters: This is a data-boundary leak. Even if it does not expose credentials, it exposes operational activity that belongs to another account.

Resolution: Queue event retrieval is now filtered by userId everywhere user-facing incidents are shown.

Impacted files: lib/server/job-queue.ts, app/api/notifications/route.ts, app/api/ops/overview/route.ts

3. Same-origin protection was inconsistent across mutating APIs

Severity: High

Problem: Several authenticated write endpoints relied only on session auth and omitted an origin check.

Why it matters: Authenticated APIs without same-origin enforcement are more exposed to CSRF-style browser abuse, especially in cookie-based auth systems.

Resolution: A shared same-origin guard is now enforced consistently across remaining mutating APIs, with human-readable 403 responses.

Impacted files: lib/server/csrf.ts, app/api/chat/route.ts, app/api/content/publish/route.ts, app/api/reports/route.ts, app/api/register/route.ts

4. The async processing layer was not durable enough for production-style automation

Severity: High

Problem: Queue work originally depended on memory-oriented execution paths without durable retries, dead-letter behavior, or lease-based claiming.

Why it matters: That makes automation brittle under crashes, restarts, and multiple workers. It also makes incident recovery much harder.

Resolution: A database-backed BackgroundJob model now stores job payloads, status, attempts, leases, retry timing, and dead-letter outcomes.

Impacted files: prisma/schema.prisma, lib/server/job-queue.ts, scripts/drain-jobs.ts

5. Reports workspace made too many client-side requests

Severity: Medium

Problem: The reports screen loaded through a six-request client waterfall instead of a single aggregated API response.

Why it matters: This increased page latency, duplicated error handling, and made the screen feel fragile when any one endpoint failed.

Resolution: A dedicated reports workspace endpoint now aggregates audits, reports, schedules, deliveries, and project counts in one call.

Impacted files: app/api/reports/workspace/route.ts, app/(dashboard)/reports/page.tsx

6. Project profile parsing could crash on partial legacy JSON

Severity: Medium

Problem: Nested JSON-like fields such as backlinkRules and contentPlaybook assumed fully formed objects and arrays.

Why it matters: Legacy or malformed records could trigger runtime crashes in the Projects UI and downstream AI context builders.

Resolution: Normalization helpers now safely parse arrays and nested objects, defaulting invalid shapes to empty, typed values.

Impacted files: lib/services/project-profile-service.ts

7. Documentation drifted behind the codebase

Severity: Medium

Problem: SRS, ERD, and architecture docs still described several integrations and queue behaviors as future work after they had already been implemented.

Why it matters: When documentation lags behind the code, onboarding, auditing, and future architecture decisions become error-prone.

Resolution: README, SRS, Architecture, ERD, and Features docs were updated to reflect the current queue model, integrations, and data structures.

Impacted files: README.md, docs/SRS.md, docs/ARCHITECTURE.md, docs/ERD.md, docs/FEATURES.md

8. There was not enough browser-level regression coverage for critical dashboards

Severity: Medium

Problem: The project had strong lint/build checks, but lacked real browser verification for reports, projects, and publishing flows.

Why it matters: Pure unit/build checks do not catch real UI breakages, rendering mismatches, or cross-route interaction regressions.

Resolution: A Playwright suite now covers Projects, Reports, and Content flows with a dedicated mock CMS endpoint for safe test execution.

Impacted files: playwright.config.ts, tests/browser/projects.spec.ts, tests/browser/reports.spec.ts, tests/browser/content.spec.ts

3. Error Fixing

1. Prevent duplicate recurring reports after worker interruption

Context: The schedule lease logic needed to move nextRunAt forward as soon as the due schedule was claimed.

Before

```
data: {
  leaseUntil,
  processingRunAt:
schedule.nextRunAt,
}
```

After

```
data: {
  leaseUntil,
  processingRunAt:
schedule.nextRunAt,
  nextRunAt,
}
```

Outcome: The next schedule window is reserved immediately, so the same report run is not re-queued after a lease timeout.

2. Replace reports-page request waterfall with one workspace API

Context: The reports UI originally made several independent requests and stitched them together client-side.

Before

```
const [auditsRes, reportsRes,
schedulesRes, deliveriesRes,
projectsRes, agentStatsRes] =
  await Promise.all([...six
fetches...]);
```

After

```
const response = await
fetch("/api/reports/workspace");
const payload = await
response.json();
```

Outcome: The page is faster, failure handling is simpler, and the API boundary is easier to evolve.

3. Normalize nested project memory payloads safely

Context: Project memory fields could contain partial or legacy JSON that did not match the latest UI shape.

Before

```
backlinkRules:
JSON.parse(profile.backlinkRules),
contentPlaybook:
JSON.parse(profile.contentPlaybook),
```

After

```
backlinkRules:
parseBacklinkRules(profile.backlinkRules),
contentPlaybook:
parseContentPlaybook(profile.contentPlaybook),
nichePlaybook:
parseNichePlaybook(profile.nichePlaybook),
```

Outcome: Projects no longer crash when an older record is missing arrays or nested properties.

4. Introduce durable queue semantics instead of memory-only job execution

Context: Async processing needed production-safe retry and recovery behavior.

Before

```
queue.push(job);
setTimeout(() => runJob(job), 0);
```

After

```
await prisma.backgroundJob.create({
  data: {
    userId,
    jobName: job.name,
    payload:
JSON.stringify(job.payload),
    status: "pending",
    maxAttempts:
getJobMaxAttempts(job.name),
  },
});
```

Outcome: Background work survives restarts, supports leasing, and exposes dead-letter state for recovery.

4. Functional Improvements Implemented

UI/UX

Backend Logic

- Reports now load through a faster workspace endpoint, reducing spinner churn and split-failure states.
- Ops Center exposes processing and dead-letter jobs more clearly, which improves operator trust in automation.
- Projects integration settings now support connection diagnostics instead of silent failure later in the workflow.
- Register UI now matches backend password policy so operators are not blocked by mismatched validation rules.

- Queue processing is now durable and resumable, with clear status transitions and final-failure cleanup behavior.
- Route handlers consistently use shared auth, response, validation, CSRF, and observability helpers.
- Report schedule execution separates claim time from completion time, making retries and failure semantics safer.
- Project services now normalize structured JSON fields before UI or AI layers consume them.

Database

- Added a BackgroundJob model to persist async work and worker leases.
- Retained targeted indexes for audits, reports, chat sessions, telemetry, backlink prospects, and schedules.
- Kept a Postgres-ready schema workflow while preserving SQLite for local development.
- Improved schema documentation so data relationships remain auditable as the platform grows.

API Structure

- Created `/api/reports/workspace` to replace a multi-endpoint waterfall with one structured payload.
- Standardized same-origin checks on remaining write endpoints.
- Added route-level timing instrumentation for heavy dashboards and workspace APIs.
- Improved error handling so failures surface in operator-friendly language instead of raw technical noise.

5. Feature Enhancements Implemented

- Durable background job system with retries, leases, and dead-letter handling.
- Project integration diagnostics for Search Console, GA4, and CMS settings.
- Browser regression suite for Projects, Reports, and Content workflows.

6. Performance Optimization

- Parallelized high-value dashboard query groups with Promise.all where the data is independent.
- Removed a client-side waterfall from the reports workspace.
- Added slow-route timing instrumentation so expensive APIs can be measured instead of guessed about.
- Preserved indexed access paths for schedule polling, chat ordering, telemetry windows, and backlink pipeline views.
- Shifted automation state out of memory-only execution into a queryable durable store.

Important Database Indexes

- Audit(userId, createdAt desc)
- Audit(userId, status, createdAt desc)
- ChatSession(userId, updatedAt desc)
- ReportSchedule(isActive, nextRunAt, leaseUntil)
- BacklinkProspect(userId, stage, createdAt desc)
- BacklinkProspect(campaignId, linkAcquired)
- AIProviderEvent(userId, providerId, createdAt)
- BackgroundJob(status, availableAt, leaseUntil)

7. Code Structure Improvement

- Validation concerns are now centralized under lib/validation instead of repeated inline in routes.
- Operational helpers such as CSRF, observability, and queue logic are now reusable modules instead of ad hoc logic.

- Documentation now mirrors the actual architecture, which reduces future design drift.
- The repo can now regenerate an executive engineering review package from source data and a build script.

8. Final Summary

Fixes Applied

- Implemented a durable database-backed queue with retries, lease handling, and dead-letter behavior.
- Fixed schedule claiming so recurring reports cannot silently duplicate after worker interruption.
- Scoped user-visible job incidents per operator account.
- Expanded same-origin enforcement across mutating APIs.
- Consolidated the reports screen into a single aggregated workspace endpoint.
- Hardened project profile parsing for partial and malformed nested data.
- Corrected documentation drift so architecture and schema docs match the running codebase.
- Added browser-level regression coverage for critical operator workflows.

Verification Completed

- pnpm exec prisma db push
- pnpm exec prisma generate
- pnpm db:generate:postgres
- pnpm test
- pnpm lint
- pnpm exec tsc --noEmit
- pnpm test:browser
- pnpm build

Suggested Next Steps

- Promote PostgreSQL as the production database default and document backup/migration procedures.
- Move queue execution to a shared Redis or workflow engine once the platform runs across multiple application instances.
- Add OAuth-style onboarding for Google integrations so operators do not have to manage service-account JSON manually.
- Introduce RBAC and approval flows if the platform expands from a single operator to multiple team members.
- Add cost analytics for AI providers, email delivery, and automation throughput so the Ops layer can optimize for ROI as well as uptime.